

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
(МОСКОВСКИЙ ПОЛИТЕХ)

КУРСОВОЙ ПРОЕКТ

по курсу
«Разработка веб-приложений»

ТЕМА

«Разработка веб-приложения для учёта и контроля приёма
лекарств»

Выполнил: Соловьева Анастасия Николаевна

Группа: 241-326

Проверил: Кружалов Алексей Сергеевич

Москва, 2026

**«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
(МОСКОВСКИЙ ПОЛИТЕХ)**

УТВЕРЖДАЮ
заведующая кафедрой
«Инфокогнитивные технологии»

_____ / Е. А. Пухова /

«_____» _____ 2026 г.

ЗАДАНИЕ

на выполнение курсовой работы (проекта)

Соловьевой Анастасии Николаевне,

обучающейся группы 241-326,

направления подготовки 09.03.01 «Информатика и вычислительная техника»

по дисциплине «Разработка веб-приложений»

на тему «Разработка веб-приложения для учёта и контроля приёма лекарств»

1. Исходные данные к работе (проекту): информационные ресурсы в сети интернет, научные публикации в открытой печати.

2. Содержание задания по курсовой работе (проекту) – перечень вопросов, подлежащих разработке:

Разрабатываемый вопрос	Объем, %	Срок	Примечание
Раздел 1. Анализ предметной области			
Задача 1.1. Обзор существующих программных продуктов для учёта и контроля приёма лекарств	8	06.04.2026	
Задача 1.2. Анализ программных инструментов разработки веб-приложения	6	12.04.2026	
Задача 1.3. Формулировка цели и задач работы	6	18.04.2026	
Раздел 2. Проектирование веб-приложения			
Задача 2.1. Анализ целевой аудитории	5	22.04.2026	
Задача 2.2. Описание функциональности приложения (диаграмма вариантов использования, user story)	7	28.04.2026	

Разрабатываемый вопрос	Объем, %	Срок	Примечание
Задача 2.3. Проектирование модели данных (ER-диаграмма, логическая и физическая схемы БД)	8	06.05.2026	
Задача 2.4. Разработка макетов страниц (Wireframe)	5	16.05.2026	
Раздел 3. Разработка веб-приложения			
Задача 3.1. Разработка базовой структуры приложения и вёрстка шаблонов страниц	10	22.05.2026	
Задача 3.2. Реализация системы аутентификации и разграничения прав доступа по ролям	7	26.05.2026	
Задача 3.3. Реализация CRUD-интерфейса для управления лекарствами и курсами лечения	10	30.05.2026	
Задача 3.4. Реализация системы учёта приёмов лекарств и агрегации статистики	12	03.06.2026	
Задача 3.5. Реализация загрузки и выгрузки файлов	6	06.06.2026	
Раздел 4. Оформление итогов работы			
Задача 4.1. Создание Git-репозитория с кодом проекта	3	02.04.2026	
Задача 4.2. Деплой приложения на хостинг	4	10.06.2026	
Задача 4.3. Оформление отчёта о проделанной работе	3	13.06.2026	

Руководитель курсовой работы (проекта):
старший преподаватель кафедры «Инфокогнитивные технологии»

«_____» _____ 2026 г. _____ А. С. Кружалов
(подпись)

Дата выдачи задания «02» апреля 2026 г.

Дата сдачи выполненной работы «13» июня 2026 г.

Задание принял к исполнению

«02» апреля 2026 г.  А. Н. Соловьева
(подпись)

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	5
ГЛАВА 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ	6
1.1 Обзор существующих программных продуктов для учета и контроля приема лекарств	6
1.2 Анализ программных инструментов разработки веб-приложения	7
1.3 Формулировка цели и задач работы	10
ГЛАВА 2. ПРОЕКТИРОВАНИЕ ВЕБ-ПРИЛОЖЕНИЯ	12
2.1 Анализ целевой аудитории	12
2.2 Описание функциональности приложения	14
2.3 Проектирование модели данных	17
2.4 Разработка макетов страниц	21
ГЛАВА 3. РАЗРАБОТКА ВЕБ-ПРИЛОЖЕНИЯ	26
3.1 Разработка базовой структуры приложения и вёрстка шаблонов страниц	26
3.2 Реализация системы аутентификации и разграничения прав доступа по ролям	30
3.3 Реализация CRUD-интерфейса для управления лекарствами и курсами лечения	32
3.4 Реализация системы учёта приёмов лекарств и агрегация ста- тистики	35
3.5 Реализация загрузки и выгрузки файлов	39
ЗАКЛЮЧЕНИЕ	43

ВВЕДЕНИЕ

Актуальность темы. В современной медицинской практике эффективность терапии напрямую зависит от приверженности пациента назначенному лечению (комплаентности). По данным ВОЗ, около 50% пациентов с хроническими заболеваниями не соблюдают график приема препаратов, что ведет к осложнениям и росту затрат на здравоохранение [1]. Существующие решения зачастую ориентированы только на самоконтроль пациента, исключая врача из процесса оперативного мониторинга. Разработка веб-приложения, объединяющего функционал ведения дневника приема с инструментами врачебного контроля и автоматизированного анализа конфликтов лекарственных средств, является актуальной задачей, направленной на повышение качества терапевтического процесса.

Степень разработанности проблемы. На рынке представлено значительное количество решений для контроля приема лекарств (например, Medisafe, MyTherapy), однако большинство из них являются закрытыми проприетарными системами без возможности интеграции с локальными медицинскими базами данных. В российском сегменте наблюдается дефицит веб-ориентированных платформ, поддерживающих полноценную двухролевую модель взаимодействия «врач – пациент» и экспорт отчетности в формате, пригодном для включения в медицинскую карту (PDF).

Объект исследования – процесс информационного сопровождения и контроля медикаментозного лечения пациентов в амбулаторных условиях.

Предмет исследования – методы и программные средства для автоматизации мониторинга приема лекарственных средств на базе современных веб-технологий.

Целью данной работы является проектирование и программная реализация веб-приложения для учета и контроля приема лекарственных средств, обеспечивающего двустороннее взаимодействие между врачом и пациентом и автоматизацию анализа приверженности лечению.

ГЛАВА 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 Обзор существующих программных продуктов для учета и контроля приема лекарств

Соблюдение режима медикаментозного лечения является одним из ключевых факторов его эффективности. В ответ на эту проблему рынок программного обеспечения предлагает широкий спектр решений, ориентированных на поддержку пациентов и медицинских специалистов.

Medisafe (США) – популярное мобильное решение для iOS и Android, позволяющее формировать графики приема, отслеживать взаимодействие лекарств и экспортировать отчеты в PDF/Excel [2]. Система поддерживает интеграцию с Apple HealthKit, ведение семейных профилей и уведомления о пополнении запасов препаратов. К ключевым недостаткам относятся отсутствие веб-интерфейса для врача, лимиты на количество лекарств в бесплатной версии и невозможность дистанционного назначения курсов лечения специалистом.

MyTherapy (Германия) – бесплатный цифровой дневник, обеспечивающий документирование приемов лекарств, симптомов и медицинских измерений [3]. Система позволяет формировать детализированные отчеты, однако доступ к данным ограничен интерфейсом самого пациента. Ключевыми недостатками являются отсутствие выделенной роли врача и механизмов контроля совпадения времени приема различных препаратов.

«Приём лекарств» (Россия) – ориентировано на составление гибкого расписания, поддерживающего в том числе сложные схемы с чередованием дней приёма и пропуска [4]. К основным недостаткам относятся отсутствие системы ролей, что делает невозможным дистанционный мониторинг со стороны врача, отсутствует аналитика соблюдения курса, также нет возможности формирования отчетов.

«Мои таблетки» (Россия) – реализует напоминания о приёме с гибкой настройкой дозировки и быстрое повторение прошедших курсов [5]. Основные недостатки включают отсутствие ролевого интерфейса врача, инструментов статистического анализа комплаентности и функциональности экспорта отчетов. Это делает систему закрытой и непригодной для дистанционного взаимодействия пациента со специалистом.

В таблице 1.1 представлен сравнительный анализ рассмотренных решений.

Таблица 1.1 – Сравнительный анализ существующих программных продуктов

Критерий	Medisafe	MyTherapy	Приём лекарств	Мои таблетки	Проект
Роль врача	–	–	–	–	+
Роль пациента	+	+	+	+	+
Назначение курсов врачом	–	–	–	–	+
Дневник приемов	+	+	+	+	+
Фото рецептов	–	–	–	–	+
Расчет % соблюдения	–	–	–	–	+
Статистика для врача	–	–	–	–	+
Предупреждение о совпадении времени	+	–	–	–	+
Экспорт истории в PDF	+	–	–	–	+
Комментарий врача к курсу	–	–	–	–	+

Сравнительный анализ показал, что ни одно из рассмотренных решений не реализует двухролевою модель взаимодействия «врач – пациент», не поддерживает прикрепление фотографий рецептов, а российские аналоги не обеспечивают экспорт истории приёмов в PDF.

Таким образом, проектируемая система представляет собой комплексный инструмент с двумя ролями: «Врач» – назначение курсов лечения и мониторинг статистики пациентов; «Пациент» – ведение дневника приёма с отметкой доз и прикреплением фотографий рецептов. Система автоматически рассчитывает процент соблюдения курса, фиксирует пропуски, формирует историю приёмов за выбранный период, предупреждает о пересечении времени приёма препаратов и обеспечивает экспорт истории в PDF.

1.2 Анализ программных инструментов разработки веб-приложения

Выбор технологического стека для реализации системы учета и контроля приема лекарств обусловлен необходимостью обеспечения высокой скорости обработки данных, надежности хранения информации и масштабируемости приложения.

Выбор языка программирования для серверной части. В таблице 1.2 представлено сравнение языков программирования для реализации серверной части приложения.

Таблица 1.2 – Сравнение языков программирования для backend

Критерий	Python	Java	Node.js
Скорость разработки	Высокая	Низкая	Средняя
Экосистема библиотек	Развитая	Развитая	Развитая
Порог вхождения	Низкий	Высокий	Средний
Поддержка веб-фреймворков	Flask, Django, FastAPI	Spring	Express
Типизация	Динамическая	Статическая	Динамическая
Применимость для учебного проекта	Высокая	Низкая	Средняя

Python 3.13 выбран благодаря низкому порогу вхождения, высокой скорости разработки и богатой экосистеме библиотек [6].

Выбор веб-фреймворка для серверной части. В таблице 1.3 представлено сравнение веб-фреймворков на языке Python.

Таблица 1.3 – Сравнение веб-фреймворков

Критерий	Flask	Django	FastAPI
Тип	Микрофреймворк	Полнофункциональный	Async-фреймворк
Встроенная ORM	–	+	–
Административная панель	–	+	–
Гибкость архитектуры	Высокая	Низкая	Средняя
Серверный рендеринг страниц	+	+	Требует настройки
Сложность освоения	Низкая	Средняя	Средняя
Применимость для проекта	Высокая	Избыточен	Избыточен

Flask 3.1.3 выбран как микрофреймворк, обеспечивающий необходимую гибкость архитектуры без излишней сложности. Django признан избыточным ввиду наличия встроенных компонентов, не требующихся в данном проекте. FastAPI ориентирован на асинхронные API-сервисы и требует дополнительной настройки для серверного рендеринга страниц [7].

Выбор системы управления базами данных. В таблице 1.4 представлено сравнение СУБД.

Таблица 1.4 – Сравнение систем управления базами данных

Критерий	SQLite	PostgreSQL	MySQL
Требует отдельного сервера	–	+	+
Транзакционность	+	+	+
Целостность данных	+	+	+
Подходит для малого числа пользователей	+	–	–
Сложность развёртывания	Низкая	Высокая	Средняя
Применимость для учебного проекта	Высокая	Избыточен	Избыточен

SQLite 3 выбран как встраиваемая СУБД, не требующая отдельного сервера и обеспечивающая транзакционность и целостность данных. Для учебного проекта с ограниченным числом пользователей это оптимальное решение. PostgreSQL и MySQL избыточны ввиду сложности развёртывания [8].

Выбор инструментов для клиентской части. В таблице 1.5 представлено сравнение подходов к реализации клиентской части.

Таблица 1.5 – Сравнение инструментов для frontend

Критерий	HTML5/CSS3/JS + Bootstrap	React	Vue.js
Требует отдельной сборки	–	+	+
Серверный рендеринг (SSR)	+	Требует настройки	Требует настройки
Сложность освоения	Низкая	Высокая	Средняя
Готовые UI-компоненты	Bootstrap 5	Сторонние библиотеки	Сторонние библиотеки
Интеграция с Jinja2 и Flask	+	Затруднена	Затруднена
Применимость для проекта	Высокая	Избыточен	Избыточен

Классический стек HTML5, CSS3, JavaScript ES6+ в связке с Bootstrap 5 [9] и шаблонизатором Jinja2 [10] выбран как наиболее подходящий для данного проекта. SPA-фреймворки React и Vue.js требуют отдельной сборки и усложняют интеграцию с серверным рендерингом Flask, что делает их избыточными в рамках курсового проекта.

Подобный технологический стек гарантирует отзывчивость интерфейса и согласованность отображения медицинских данных для обеих ролей пользователей.

1.3 Формулировка цели и задач работы

На основе проведённого анализа существующих программных продуктов и выявленных функциональных пробелов в существующих решениях была сформулирована цель проекта.

Цель проекта – разработать веб-приложение для учёта и контроля приёма лекарств, реализующее двухролевою модель взаимодействия «врач – пациент» и обеспечивающее автоматизированный мониторинг соблюдения курса лечения.

Для достижения поставленной цели необходимо решить следующие задачи:

1. Провести сравнительный анализ существующих программных продуктов для учёта и контроля приёма лекарств и обосновать выбор технологического стека разрабатываемого приложения.
2. Спроектировать архитектуру веб-приложения и разработать структуру базы данных для хранения сведений о пользователях, курсах лечения и истории приёмов.
3. Разработать макеты страниц и описать функциональность приложения посредством диаграммы вариантов использования.
4. Реализовать систему аутентификации и разграничения прав доступа по ролям «Врач» и «Пациент».
5. Разработать модули управления лекарственными препаратами и курсами лечения.
6. Реализовать дневник приёма лекарств с возможностью отметки принятых доз и прикрепления фотографий рецептов.
7. Реализовать автоматический расчёт статистики соблюдения курса лечения и механизм предупреждения о пересечении времени приёма препаратов.
8. Обеспечить экспорт истории приёмов в формат PDF.

9. Выполнить функциональное тестирование разработанного приложения, развернуть систему на сервере и провести оценку соответствия реализованных модулей поставленным требованиям.

ГЛАВА 2. ПРОЕКТИРОВАНИЕ ВЕБ-ПРИЛОЖЕНИЯ

2.1 Анализ целевой аудитории

Проектируемое веб-приложение ориентировано на две категории пользователей: пациентов и врачей. Для каждой категории сформулирован профиль целевого пользователя и выявлены ключевые потребности, определяющие функциональные требования к системе.

Профиль пользователя: Пациент. Основной целевой пользователь – человек, проходящий медикаментозное лечение в амбулаторных условиях. К данной категории относятся:

- пациенты с хроническими заболеваниями (сахарный диабет, гипертония, заболевания щитовидной железы), принимающие несколько препаратов одновременно по долгосрочным схемам лечения;
- пациенты, проходящие курсовое лечение (антибиотики, противовирусные препараты), для которых критично соблюдение точного расписания приёма;
- пожилые пациенты с высоким риском пропуска доз ввиду большого количества назначенных препаратов;
- пользователи, самостоятельно контролирующие приём витаминов и профилактических препаратов без назначения врача.

В таблице 2.1 представлен профиль целевого пользователя роли «Пациент».

Таблица 2.1 – Профиль целевого пользователя: Пациент

Характеристика	Описание
Кто	Человек, проходящий медикаментозное лечение в амбулаторных условиях
Основная цель	Соблюдение назначенного курса лечения и контроль собственного режима приёма препаратов
Ключевые потребности	Фиксация факта приёма с указанием дозировки и времени; прикрепление фотографии рецепта; предупреждения о совпадении времени приёма; самостоятельное добавление курсов; экспорт истории приёмов в PDF
Основные риски	Пропуск доз; одновременный приём несовместимых по времени препаратов; потеря информации о назначениях

Профиль пользователя: Врач. Вторая категория пользователей — медицинский специалист, ведущий пациентов в амбулаторных условиях. К данной категории относятся терапевты, врачи узких специальностей и медицинские работники, осуществляющие дистанционный контроль соблюдения назначенной терапии.

В таблице 2.2 представлен профиль целевого пользователя роли «Врач».

Таблица 2.2 – Профиль целевого пользователя: Врач

Характеристика	Описание
Кто	Медицинский специалист, ведущий пациентов в амбулаторных условиях
Основная цель	Контроль приверженности пациентов назначенной терапии и своевременная коррекция лечения
Ключевые потребности	Назначение курсов лечения; дистанционный мониторинг статистики соблюдения курса; просмотр истории приёмов и зафиксированных пропусков; направление комментариев пациенту; экспорт отчётности в PDF
Основные риски	Низкая приверженность пациента лечению; отсутствие оперативной обратной связи между врачом и пациентом

На основе выявленных потребностей каждой категории пользователей в таблице 2.3 представлено сопоставление профилей целевых пользователей.

Таблица 2.3 – Сравнение профилей целевых пользователей

Характеристика	Пациент	Врач
Основная цель	Соблюдение назначенного курса лечения	Контроль приверженности пациентов терапии
Ключевые действия	Отметка доз, прикрепление фото рецептов, просмотр истории	Назначение курсов, просмотр статистики, комментирование
Самостоятельное добавление курса	Да	Да
Потребность в аналитике	Личная статистика соблюдения курса	Агрегированная статистика по пациентам
Обратная связь	Получение комментария от врача	Отправка комментария пациенту
Экспорт данных	История приёмов в PDF	Отчётность по пациенту в PDF
Критичные функции	Предупреждение о совпадении времени приёма препаратов	Процент соблюдения, фиксация пропусков

Таким образом, проведенный анализ целевой аудитории обосновал необходимость реализации двухролевой модели взаимодействия. Четкое разделение функций между пациентом и врачом позволяет эффективно решать проблему приверженности терапии. Выявленные профили, потребности и риски пользователей служат прямым основанием для формирования функци-

ональных требований к системе, детально описанных в подразделе 2.2.

2.2 Описание функциональности приложения

В данном подразделе описана функциональность веб-приложения посредством диаграммы вариантов использования (Use Case Diagram) и пользовательских историй (User Story). Приложение реализует двухролевою модель: **Врач и Пациент**.

Система базируется на двухролевой модели взаимодействия, разделяющей зоны ответственности акторов «Пациент» и «Врач». Архитектура приложения предусматривает как изолированный функционал для каждой роли (например, ведение дневника самоконтроля пациентом или назначение терапии врачом), так и ряд общих прецедентов, обеспечивающих сквозные процессы взаимодействия. К совместным вариантам использования относятся авторизация в системе, просмотр аналитических данных, а также ретроспективный анализ истории приёмов.

Стоит отметить, что прецедент «Экспорт данных в PDF» связан с базовым вариантом использования «Просмотр истории приёмов» отношением расширения («extend»). Данное архитектурное решение обусловлено тем, что выгрузка отчётности является опциональным действием, которое инициируется пользователем по требованию и не нарушает основной сценарий просмотра данных в интерфейсе.

На рисунке 2.1 представлена диаграмма вариантов использования системы. Цветом выделены группы функций: фиолетовым – эксклюзивные функции роли «Пациент», синим — эксклюзивные функции роли «Врач», зелёным — функции, доступные обеим ролям.

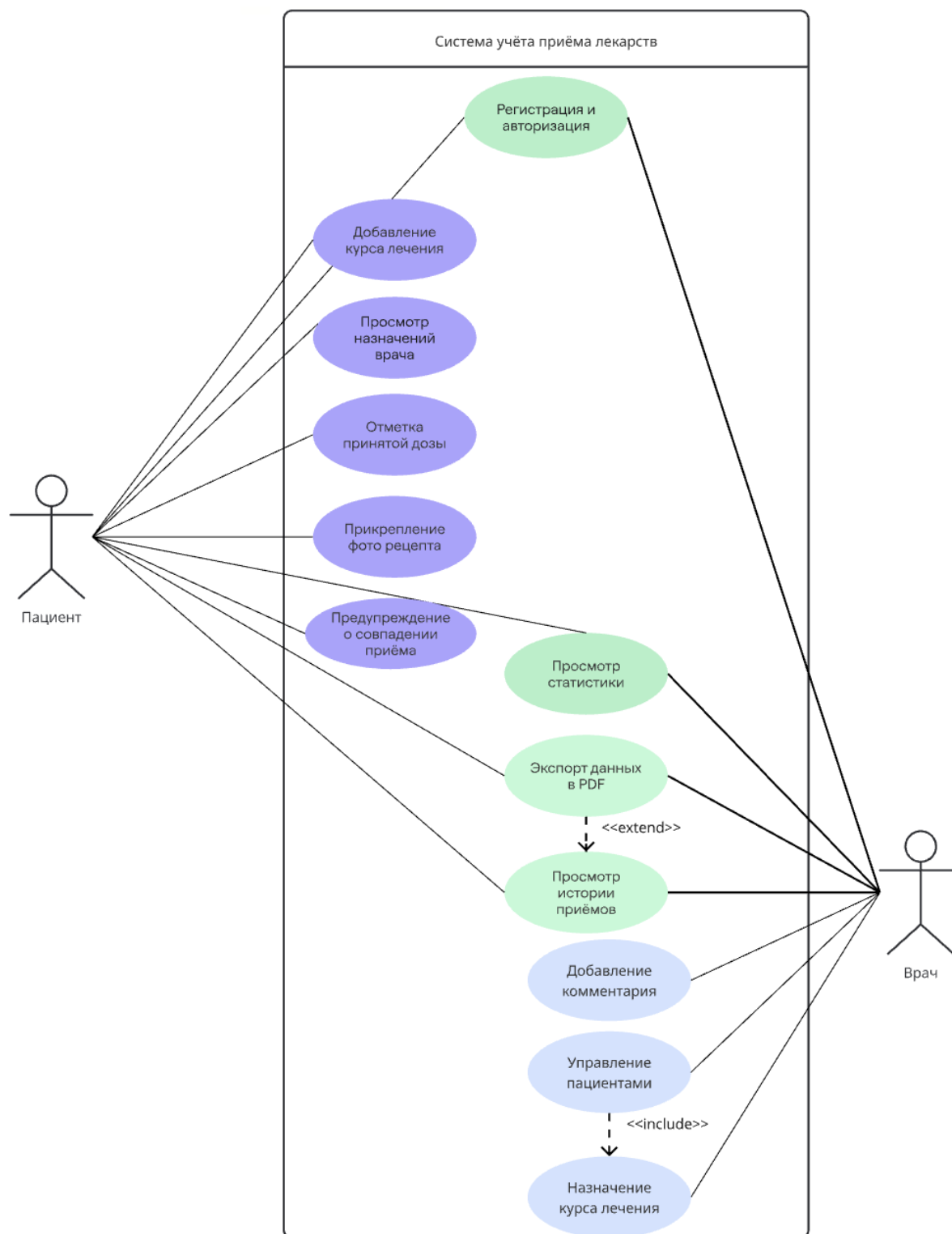


Рисунок 2.1 – Диаграмма вариантов использования

В таблице 2.4 представлены пользовательские истории (User Story) для обеих ролей.

Таблица 2.4 – Пользовательские истории (User Story)

№	Роль	Я хочу...	Чтобы...
1	Пациент	отмечать принятые дозы лекарств	контролировать соблюдение курса лечения
2	Пациент	прикреплять фото рецепта	хранить историю назначений в одном месте
3	Пациент	самостоятельно добавить курс лечения	отслеживать приём препаратов без участия врача
4	Пациент	получать предупреждения о пересечении времени приёма	избегать одновременного приёма несовместимых препаратов
5	Пациент	просматривать статистику соблюдения курса	оценивать свою приверженность лечению
6	Пациент	экспортировать историю приёмов в PDF	предоставить её врачу или включить в медицинскую карту
7	Врач	назначать курсы лечения пациентам	дистанционно управлять терапией
8	Врач	просматривать статистику соблюдения курса пациентом	оценивать приверженность лечению и корректировать терапию
9	Врач	просматривать историю приёмов пациента	отслеживать динамику лечения за выбранный период
10	Врач	экспортировать историю приёмов пациента в PDF	включить данные в медицинскую документацию
11	Врач	оставлять комментарий к курсу лечения	информировать пациента о необходимости соблюдения назначений

Таким образом, описанная логика взаимодействия акторов и структура обрабатываемых данных выступают основой для следующего этапа разработки — проектирования таблиц реляционной базы данных.

2.3 Проектирование модели данных

На основе анализа функциональных требований разработана модель данных веб-приложения, включающая концептуальную (ER-диаграмму), логическую и физическую схемы базы данных.

ER-диаграмма. Концептуальная модель данных, отражающая информационную структуру предметной области веб-приложения, представлена в виде ER-диаграммы на рисунке 2.2.

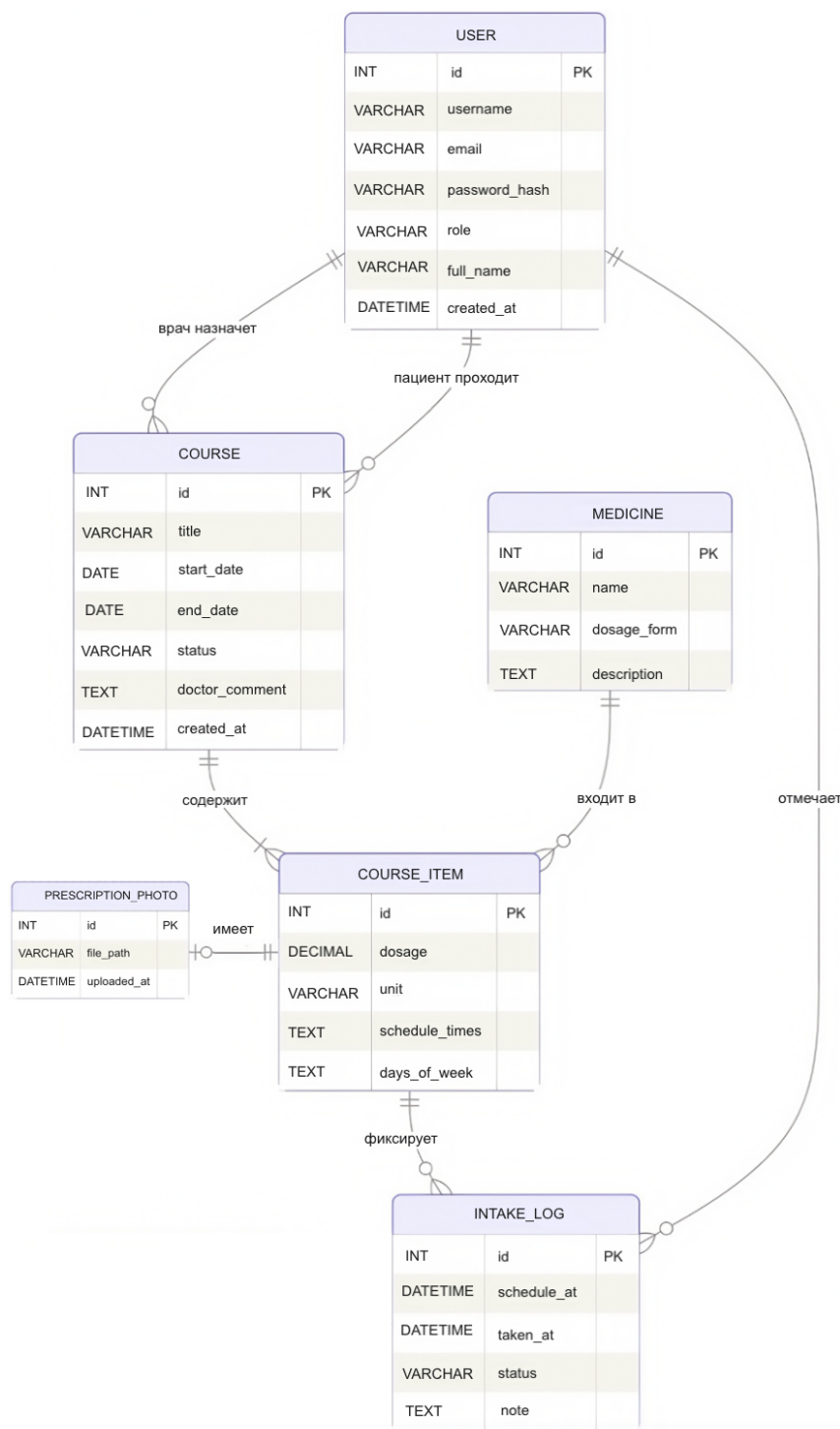


Рисунок 2.2 – ER-диаграмма базы данных

Архитектура базы данных базируется на шести ключевых сущностях:

1. **USER** – пользователи системы (пациенты и врачи).
2. **MEDICINE** – глобальный справочник лекарственных средств.
3. **COURSE** – общие сведения о курсах лечения.
4. **COURSE_ITEM** – конкретные медицинские препараты в рамках курса.
5. **INTAKE_LOG** – журнал фактического и запланированного приёма доз.
6. **PRESCRIPTION_PHOTO** – цифровые копии и фотографии рецептов.

Взаимодействие между элементами разработанной модели, детальное описание бизнес-логики, а также характер и степени связей между сущностями подробно систематизированы в таблице 2.5.

Таблица 2.5 – Спецификация связей концептуальной модели данных

№	Взаимодействующие сущности	Тип связи	Бизнес-логика и характер связи
1	USER – COURSE	1 : <i>M</i>	Врач назначает курс лечения
2	USER – COURSE	1 : <i>M</i>	Пациент самостоятельно создаёт курс лечения
3	COURSE – COURSE_ITEM	1 : <i>M</i>	Курс содержит позиции (медикаменты)
4	MEDICINE – COURSE_ITEM	1 : <i>M</i>	Препарат из глобального справочника включается в позицию курса для исключения опечаток в названиях
5	COURSE_ITEM – INTAKE_LOG	1 : <i>M</i>	Позиция курса расписывается в журнал приёмов. Генерирует атомарные отметки времени для контроля каждой дозы
6	USER – INTAKE_LOG	1 : <i>M</i>	Пациент ведёт журнал приёмов
7	COURSE_ITEM – PRESCRIPTION_PHOTO	1 : 1	Позиция курса может быть подтверждена фотографией бумажного рецепта врача

Логическая схема базы данных. Логическая схема описывает таблицы, их поля и универсальные типы данных без привязки к конкретной СУБД. Схема включает шесть таблиц: user, medicine, course, course_item, intake_log, prescription_photo. Связи между таблицами реализованы через внешние ключи (FK), отражённые в соответствующих полях. На рисунке 2.3 представлена логическая схема базы данных.

INTEGER, REAL, TEXT, VARCHAR(n). Для каждого поля определены ограничения целостности — NOT NULL, UNIQUE, DEFAULT, CHECK, а каскадные операции ON DELETE CASCADE обеспечивают автоматическое удаление зависимых записей. На рисунке 2.4 представлена физическая схема базы данных.

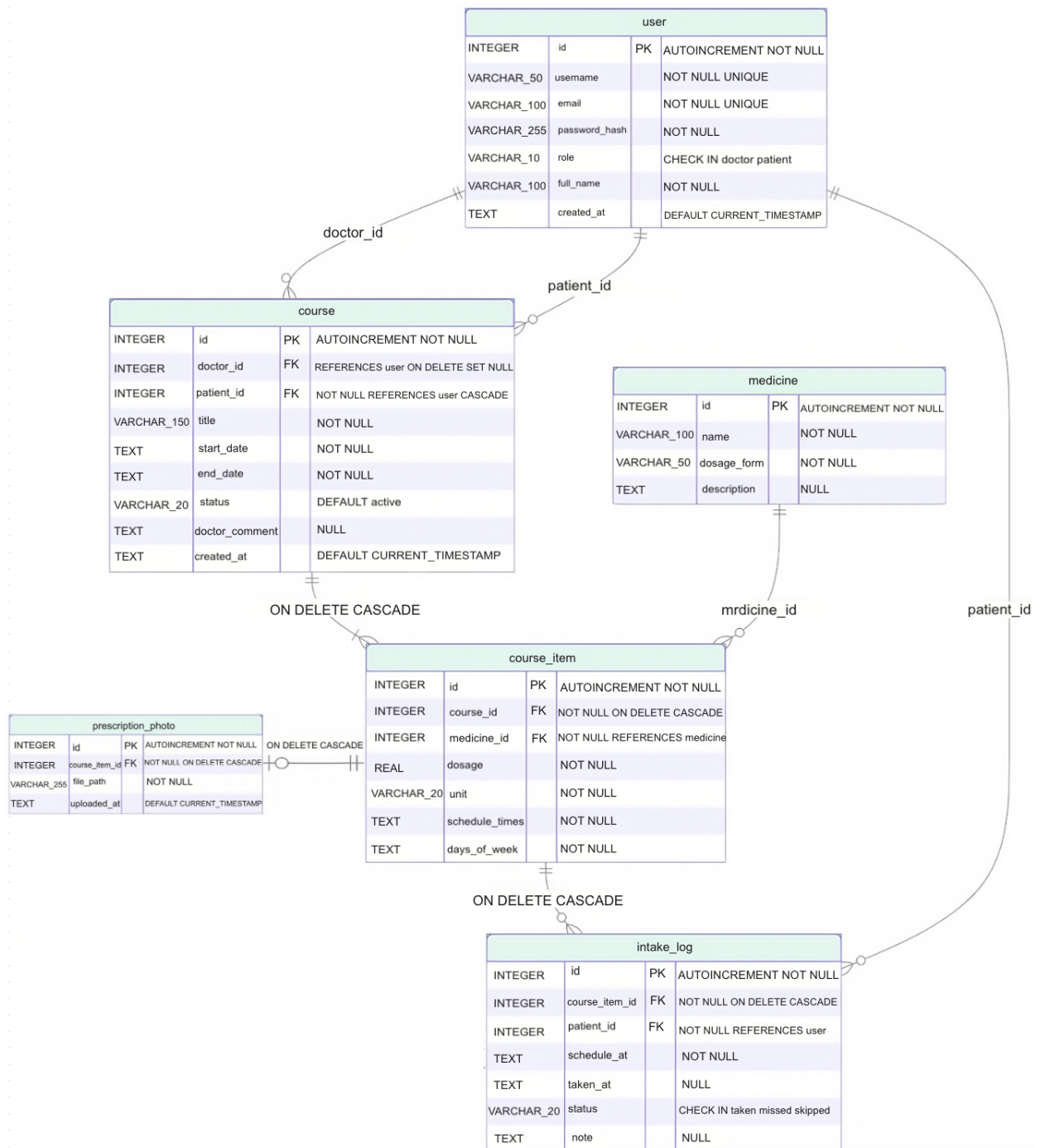


Рисунок 2.4 – Физическая схема базы данных

2.4 Разработка макетов страниц

Проектирование интерфейса веб-приложения выполнялось методом *code-based prototyping* с использованием HTML и CSS. Это позволило проработать структуру экранов с учётом реального технологического стека приложения (Flask, Jinja2, Bootstrap 5).

Комплекс макетов включает общие экраны, экраны интерфейса пациента и экраны интерфейса врача.

К **общим экранам** относятся:

- страница входа в систему (рисунок 2.5);
- страница регистрации пользователей с возможностью выбора роли (рисунок 2.6).

Интерфейс пациента включает в себя следующие экраны:

- дашборд пациента с графиком приёмов на сегодня, предупреждениями о совпадении времени приёма лекарств и сводкой по активным курсам (рисунок 2.7);
- дневник приёмов пациента с интерактивным мини-календарём и кнопками быстрого подтверждения (рисунок 2.8);
- форма самостоятельного добавления курса лечения пациентом (рисунок 2.9).

Экраны **интерфейса врача** включают:

- дашборд врача со списком закреплённых пациентов и прогресс-барами соблюдения ими терапевтических назначений (рисунок 2.10);
- форма назначения нового курса лечения врачом (рисунок 2.11);
- интерфейс добавления экспертного комментария и заметок к курсу лечения (рисунок 2.12).

The screenshot shows a web browser window with a light blue background. At the top center is a logo consisting of a circle with a stylized 'M' and 'T' inside. Below the logo, the text 'МедТрекер' is displayed in bold, followed by 'Система учёта приёма лекарств' in a smaller font. The main content area is a white rounded rectangle containing two tabs: 'Вход' (selected) and 'Регистрация'. Below the tabs are two input fields: 'Email' with the value 'user@example.com' and an envelope icon, and 'Пароль' with masked characters '.....' and an eye icon. To the right of the password field is a link 'Забыли пароль?'. Below these fields is a large black button with the text 'Войти'. At the bottom of the white rectangle is a link 'Нет аккаунта? Зарегистрироваться'.

Рисунок 2.5 – Страница входа

The screenshot shows a web browser window with a light blue background. The main content area is a white rounded rectangle containing two tabs: 'Вход' and 'Регистрация' (selected). Below the tabs are four input fields: 'Полное имя' with the value 'Иванов Иван Иванович' and a person icon, 'Email' with the value 'user@example.com' and an envelope icon, 'Пароль' with masked characters '.....' and an eye icon, and 'Роль'. The 'Роль' section has two buttons: 'Врач' with a stethoscope icon and 'Пациент' with a person icon. Below these is a large black button with the text 'Создать аккаунт'. At the bottom of the white rectangle is a link 'Уже есть аккаунт? Войти'.

Рисунок 2.6 – Страница регистрации

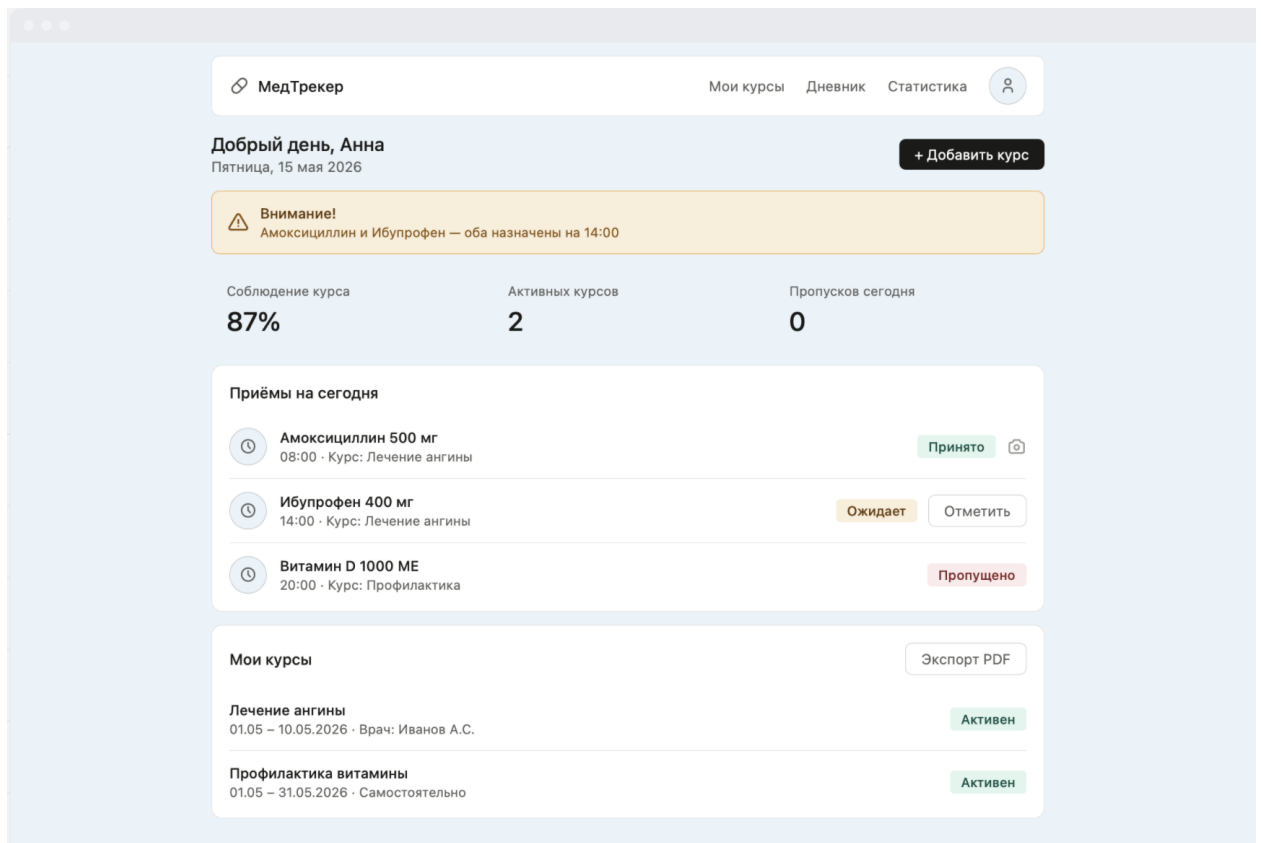


Рисунок 2.7 – Дашборд пациента

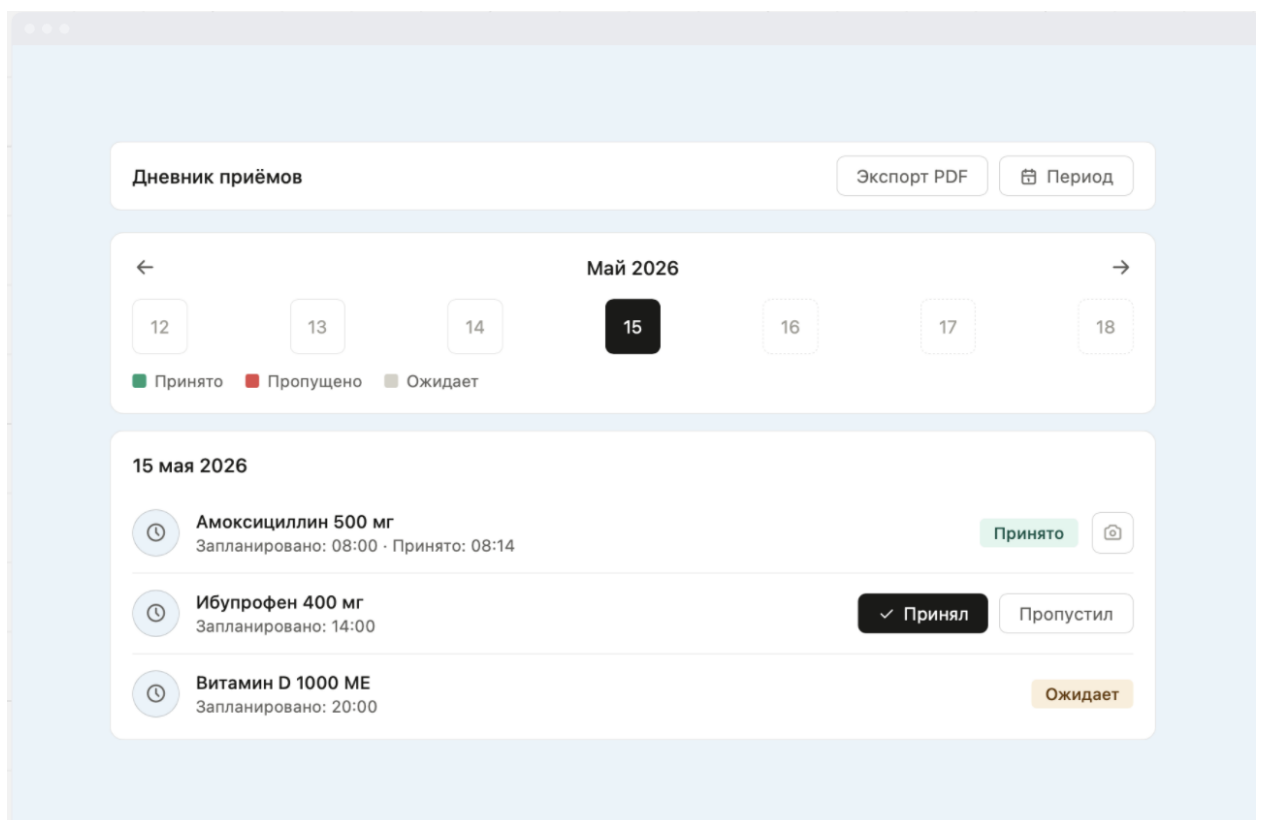


Рисунок 2.8 – Дневник приёмов пациента

← Назад к дашборду

Добавить курс лечения

❗ Самостоятельный курс

Вы добавляете курс самостоятельно. Если курс назначен врачом — он появится автоматически в вашем списке.

Основные данные

Название курса

Например: Витамин D, профилактика

Цель приёма

Краткое описание цели курса (необязательно)...

Дата начала

16.05.2026

Дата окончания

16.06.2026

Препараты

+ Добавить

Витамин D3

1000 МЕ · 1хдень

09:00

✎

🗑

Омега-3

1000 мг · 2хдень

08:00, 20:00

✎

🗑

+ Добавить препарат

Рисунок 2.9 – Добавление курса

МедТрекер

Пациенты

Назначения

Статистика

ИВ

Мои пациенты

Иванов А.С. · Терапевт

+ Назначить курс

Всего пациентов

12

Активных курсов

8

Низкое соблюдение

3

Список пациентов

🔍 Поиск

Фильтр

АС

Смирнова А.В.

Курс: Гипертония · до 30.06.2026

92%

Хорошо

Комментарий

Открыть

МК

Козлов М.Н.

Курс: Диабет · до 15.07.2026

61%

Внимание

Комментарий

Открыть

ОП

Петрова О.И.

Курс: Антибиотики · до 20.05.2026

38%

Критично

Комментарий

Открыть

Рисунок 2.10 – Дашборд врача

← Назад к пациентам **Новое назначение**

Основные данные

Пациент
Смирнова Анна Васильевна

Название курса
Лечение гипертонии

Дата начала
15.05.2026

Дата окончания
15.08.2026

Препараты курса + Добавить

Лизиноприл	10 мг · 1хдень	08:00	🗑
Амлодипин	5 мг · 1хдень	20:00	🗑

+ Добавить препарат

Сохранить назначение

Рисунок 2.11 – Назначение курса

← Назад к пациентам **Козлов М.Н. — Курс лечения** Экспорт PDF

Соблюдение курса **38%** Принято доз **14** Пропущено доз **23** Статус **Критично**

Курс: Антибиотики · 01.05 – 20.05.2026

Амоксициллин 500 мг
3 раза в день · 08:00, 14:00, 20:00 38% Критично

Комментарий к курсу ✎ Редактировать

Иванов А.С. · 14.05.2026, 10:30
Михаил Николаевич, обратите внимание на соблюдение курса антибиотиков. Прерывание курса может привести к резистентности. Пожалуйста, принимайте препарат строго по расписанию.

Новый комментарий
Введите комментарий для пациента...

Отправить комментарий

Рисунок 2.12 – Добавление комментария

ГЛАВА 3. РАЗРАБОТКА ВЕБ-ПРИЛОЖЕНИЯ

3.1 Разработка базовой структуры приложения и вёрстка шаблонов страниц

Разработка веб-приложения выполнена на основе архитектурного шаблона MVC (Model-View-Controller), реализованного средствами микрофреймворка Flask 3.1.3. Структура проекта организована по принципу разделения ответственности: модели данных, маршрутизация запросов и шаблоны представлений вынесены в отдельные модули, что обеспечивает независимость компонентов и упрощает сопровождение кода.

В качестве инструмента взаимодействия с базой данных использована ORM-библиотека Flask-SQLAlchemy, обеспечивающая объектно-ориентированный доступ к данным и избавляющая от необходимости написания прямых SQL-запросов. Это решение позволило ускорить разработку и упростить сопровождение кода при сохранении SQLite 3 в качестве СУБД, выбранной в разделе 1.2.

Организация структуры проекта. Приложение реализовано в виде пакета Python. Точка входа – файл `run.py`; конфигурация среды вынесена в `config.py`. Внутри пакета `app` выделены подпакеты `models` (ORM-модели), `routes` (маршруты по ролям), `static` (CSS и JavaScript) и `templates` (Jinja2-шаблоны). Такое разделение соответствует принципу единственной ответственности и позволяет изменять любой слой приложения независимо от остальных.

Инициализация приложения реализована через паттерн «фабрика приложения» (*application factory*), что позволяет создавать несколько экземпляров с различной конфигурацией. Маршруты зарегистрированы посредством механизма Blueprint фреймворка Flask, обеспечивающего логическое разделение функциональности по ролям пользователей. В листинге 1 представлена реализация фабрики приложения.

Листинг 1 – Фабрика приложения (`app/__init__.py`)

```
1 def create_app():
2     app = Flask(__name__, static_folder='static',
3                 template_folder='templates')
4     app.config.from_object(Config)
5
6     db.init_app(app)
```

```

7
8     login_manager = LoginManager()
9     login_manager.init_app(app)
10    login_manager.login_view = 'auth.login'
11
12    @login_manager.user_loader
13    def load_user(user_id):
14        return User.query.get(int(user_id))
15
16    from app.routes.auth      import auth_bp
17    from app.routes.patient  import patient_bp
18    from app.routes.doctor   import doctor_bp
19
20    app.register_blueprint(auth_bp)
21    app.register_blueprint(patient_bp)
22    app.register_blueprint(doctor_bp)
23
24    with app.app_context():
25        db.create_all()
26        _seed_medicines()
27
28    return app

```

При запуске приложения вызывается `db.create_all()`, который автоматически создаёт все таблицы базы данных на основе объявленных ORM-моделей, а функция `_seed_medicines()` выполняет начальное заполнение справочника лекарственных средств, если он пуст.

Реализация ORM-моделей. Модели данных реализованы с использованием Flask-SQLAlchemy и соответствуют физической схеме базы данных, разработанной в разделе 2.3. Каждый класс модели наследует от `db.Model` и описывает соответствующую таблицу с атрибутами, типами данных и ограничениями целостности. Реализованы шесть моделей: `User`, `Medicine`, `Course`, `CourseItem`, `IntakeLog`, `PrescriptionPhoto`. Каскадные ограничения ON DELETE CASCADE обеспечивают автоматическое удаление зависимых записей при удалении родительских объектов. Реализация системы аутентификации на основе модели `User` подробно рассмотрена в подразделе 3.2.

Алгоритм генерации расписания приёмов. Ключевым алгоритмом приложения является автоматическая генерация записей журнала приёмов (`IntakeLog`) при создании нового курса лечения. Алгоритм перебирает каждый день от даты начала до даты окончания курса, фильтрует дни по заданному

расписанию и для каждого подходящего дня создаёт запись с временными метками запланированного приёма. В листинге 2 представлена реализация данного алгоритма.

Листинг 2 – Алгоритм генерации расписания приёмов (app/utils.py)

```
1 def generate_scheduled_logs(course_item, course, patient_id)
  :
2     try:
3         times = json.loads(course_item.schedule_times)
4     except Exception:
5         times = ['08:00']
6     try:
7         days = json.loads(course_item.days_of_week)
8     except Exception:
9         days = ['mon', 'tue', 'wed', 'thu', 'fri', 'sat', 'sun']
10    day_numbers = {DAYS_MAP[d] for d in days if d in
DAYS_MAP}
11
12    current = course.start_date
13    logs_to_add = []
14
15    while current <= course.end_date:
16        if current.weekday() in day_numbers:
17            for t_str in times:
18                h, m = map(int, t_str.split(':'))
19                scheduled = datetime.combine(
20                    current,
21                    datetime.min.time().replace(hour=h,
minute=m)
22                )
23                logs_to_add.append(IntakeLog(
24                    course_item_id=course_item.id,
25                    patient_id=patient_id,
26                    scheduled_at=scheduled,
27                    status='pending'
28                ))
29                current += timedelta(days=1)
30
31    db.session.add_all(logs_to_add)
```

Поля *schedule_times* и *days_of_week* хранятся в формате JSON-строки – например, ["08:00 "20:00"] и ["mon "tue "wed "thu "fri"]. Словарь DAYS_MAP отображает трёхбуквенный код дня недели на числовое значение метода

weekday() объекта date, что позволяет эффективно сравнивать текущий день с заданным расписанием. Все сформированные записи сохраняются за одну операцию add_all, что снижает число обращений к базе данных.

Вёрстка шаблонов страниц. Клиентская часть приложения реализована с использованием шаблонизатора Jinja2, интегрированного во фреймворк Flask. Все страницы наследуют базовый шаблон *shared/base.html*, который содержит навигационную панель, контейнер для всплывающих уведомлений (flash-сообщений) и подключение статических ресурсов. Итоговый комплекс разработанных шаблонов включает 18 страниц, разделённых по ролям: 3 общих, 2 для аутентификации, 7 для интерфейса пациента и 6 для интерфейса врача.

В листинге 3 показан фрагмент базового шаблона, реализующего навигационную панель с динамическим отображением пунктов меню в зависимости от роли авторизованного пользователя.

Листинг 3 – Фрагмент базового шаблона с ролевой навигацией
(app/templates/shared/base.html)

```
1 {% if current_user.is_patient() %}
2     <a class="nav-link" href="{{ url_for('patient.dashboard') }}">
3         Мои курсы
4     </a>
5     <a class="nav-link" href="{{ url_for('patient.diary') }}">
6         Дневник
7     </a>
8     <a class="nav-link" href="{{ url_for('patient.statistics') }}">
9         Статистика
10    </a>
11 {% else %}
12    <a class="nav-link" href="{{ url_for('doctor.dashboard') }}">
13        Пациенты
14    </a>
15    <a class="nav-link" href="{{ url_for('doctor.assign_course') }}">
16        Назначения
17    </a>
18    <a class="nav-link" href="{{ url_for('doctor.statistics') }}">
```

```
19     Статистика
20   </a>
21 {% endif %}
```

Условный рендеринг навигации на уровне шаблона гарантирует, что пользователь видит только те разделы интерфейса, которые соответствуют его роли. Серверная проверка прав доступа, дополняющая визуальное разграничение, рассмотрена в подразделе 3.2.

Таким образом, разработанная базовая структура приложения обеспечивает чёткое разделение ответственности между компонентами, масштабируемость архитектуры и соответствие шаблонов страниц макетам, разработанным в подразделе 2.4.

3.2 Реализация системы аутентификации и разграничения прав доступа по ролям

Система аутентификации реализована на основе расширения Flask-Login, обеспечивающего управление пользовательскими сессиями. Разграничение прав доступа построено на ролевой модели: каждый пользователь системы имеет роль 'doctor' либо 'patient', хранящуюся в поле `role` таблицы `user`, и получает доступ только к тем маршрутам, которые соответствуют его роли.

Хранение паролей. Пароли пользователей хранятся исключительно в виде хэшей, формируемых алгоритмом PBKDF2-SHA256 функцией `generate_password_hash` из библиотеки Werkzeug. При регистрации вызывается метод `set_password`, сохраняющий хэш в поле `password_hash`. При входе метод `check_password` сравнивает введённый пароль с сохранённым хэшем без его обратного преобразования, что делает восстановление исходного пароля невозможным даже при компрометации базы данных.

Алгоритм аутентификации. Маршрут входа в систему обрабатывает POST-запрос формы: выполняет поиск пользователя по email, проверяет пароль и при успешной аутентификации создаёт сессию средствами Flask-Login. В листинге 4 представлена реализация маршрута входа.

Листинг 4 – Маршрут аутентификации (app/routes/auth.py)

```
1 @auth_bp.route('/login', methods=['GET', 'POST'])
2 def login():
3     error = None
```

```

4     if request.method == 'POST':
5         email      = request.form.get('email', '').strip()
6         password = request.form.get('password', '')
7         user = User.query.filter_by(email=email).first()
8         if user and user.check_password(password):
9             login_user(user, remember=True)
10            if user.is_doctor():
11                return redirect(url_for('doctor.dashboard'))
12                return redirect(url_for('patient.dashboard'))
13            error = 'Неверный email или пароль'
14            return render_template('auth/login.html', error=error)

```

После успешной проверки учётных данных функция *login_user* сохраняет идентификатор пользователя в подписанном cookie-файле сессии. Перенаправление выполняется в зависимости от значения поля *role*: врач направляется на дашборд управления пациентами, пациент – на дашборд приёмов. Единое сообщение об ошибке при неверном email или пароле исключает возможность перебора существующих учётных записей.

Разграничение прав доступа. Защита маршрутов реализована двух-уровневым декоратором. Первый уровень – `@login_required` из `Flask-Login` – проверяет факт аутентификации и перенаправляет неавторизованных пользователей на страницу входа. Второй уровень – пользовательские декораторы `patient_required` и `doctor_required` – проверяет соответствие роли пользователя требуемой и возвращает ответ 403 при нарушении. В листинге 5 представлена реализация декораторов контроля доступа.

Листинг 5 – Декораторы разграничения доступа по ролям
(`app/routes/patient.py`, `app/routes/doctor.py`)

```

1 def patient_required(f):
2     from functools import wraps
3     @wraps(f)
4     def decorated(*args, **kwargs):
5         if not current_user.is_authenticated \
6             or not current_user.is_patient():
7             abort(403)
8         return f(*args, **kwargs)
9     return decorated
10
11 def doctor_required(f):
12     from functools import wraps

```

```

13     @wraps(f)
14     def decorated(*args, **kwargs):
15         if not current_user.is_authenticated \
16             or not current_user.is_doctor():
17             abort(403)
18         return f(*args, **kwargs)
19     return decorated

```

Оба декоратора применяются совместно с `@login_required` к каждому защищённому маршруту, что гарантирует: маршруты Blueprint `/patient/...` недоступны врачам, а маршруты `/doctor/...` – пациентам. Попытка перейти по адресу чужой роли возвращает страницу с кодом 403. Дополнительно в маршрутах, работающих с конкретными записями базы данных, выполняется проверка принадлежности объекта текущему пользователю, что предотвращает несанкционированный доступ к чужим данным даже при знании идентификатора записи.

Таким образом, реализованная система аутентификации и авторизации обеспечивает защиту приложения на уровне сессии, маршрута и отдельной записи базы данных, что полностью соответствует задаче 4, сформулированной в разделе 1.3.

3.3 Реализация CRUD-интерфейса для управления лекарствами и курсами лечения

CRUD-интерфейс приложения охватывает две основные предметные сущности: справочник лекарственных средств (Medicine) и курсы лечения (Course) с вложенными позициями препаратов (CourseItem). Операции создания, чтения, обновления и удаления реализованы в маршрутах Blueprint-ов `patient` и `doctor` и доступны в соответствии с ролью пользователя.

Справочник лекарственных средств. Таблица `medicine` выступает глобальным справочником и используется обеими ролями при формировании состава курса. При добавлении препарата в курс система сначала ищет соответствующую запись в справочнике по наименованию; если запись отсутствует – она создаётся автоматически. Такой подход исключает дублирование лекарств и поддерживает целостность справочных данных. Начальное наполнение справочника выполняется функцией `_seed_medicines` при первом запуске приложения.

Создание курса. Операция создания курса различается по ролям. Пациент самостоятельно добавляет курс через маршрут `/patient/courses/add`: поле `doctor_id` остаётся незаполненным (NULL), а необязательное поле `goal` фиксирует цель приёма. Врач назначает курс через маршрут `/doctor/assign`: поле `doctor_id` заполняется идентификатором текущего врача, что позволяет системе однозначно определить источник назначения.

При сохранении курса маршрут последовательно выполняет следующие действия: валидирует обязательные поля (название, даты, пациент), сохраняет запись `Course` в базу данных, перебирает массив переданных препаратов и для каждого создаёт запись `CourseItem`, после чего вызывает `generate_scheduled_logs` для автоматической генерации расписания приёмов на весь период курса. Применение `db.session.flush()` перед созданием позиций обеспечивает наличие `course.id` до сохранения транзакции, что необходимо для установки внешнего ключа.

В таблице 3.1 представлены маршруты, реализующие полный цикл CRUD для сущности «Курс лечения».

Таблица 3.1 – Маршруты CRUD-интерфейса курсов лечения

Операция	Маршрут	Метод	Роль
Create	<code>/patient/courses/add</code>	POST	Пациент
Create	<code>/doctor/assign</code>	POST	Врач
Read	<code>/patient/courses</code>	GET	Пациент
Read	<code>/patient/courses/{id}</code>	GET	Пациент
Read	<code>/doctor/patients/{id}</code>	GET	Врач
Update	<code>/patient/courses/{id}/edit</code>	POST	Пациент
Update	<code>/doctor/courses/{id}/comment</code>	POST	Врач
Delete	<code>/patient/courses/{id}/delete</code>	POST	Пациент
Delete	<code>/doctor/courses/{id}/delete</code>	POST	Врач

В листинге 6 представлен ключевой фрагмент создания курса с автоматической генерацией расписания приёмов.

Листинг 6 – Фрагмент создания курса с генерацией расписания
(app/routes/patient.py)

```
1 course = Course(patient_id=current_user.id,
2                 doctor_id=None, title=title,
3                 start_date=start_date, end_date=end_date,
4                 goal=goal or None)
5 db.session.add(course)
6 db.session.flush()      # получаем course.id до commit()
7
8 item = CourseItem(course_id=course.id,
9                  medicine_id=medicine.id,
10                 dosage=float(dosages[i]),
11                 unit=units[i],
12                 schedule_times=times_raw,
13                 days_of_week=days_raw)
14 db.session.add(item)
15 db.session.flush()
16 generate_scheduled_logs(item, course, current_user.id)
17
18 db.session.commit()
```

Вызов `db.session.flush()` принудительно отправляет INSERT в базу данных внутри открытой транзакции, не завершая её – это необходимо для получения `course.id` до `commit()`, поскольку без него невозможно установить внешний ключ `CourseItem.course_id`.

Чтение и фильтрация данных. Выборка курсов выполняется с фильтрацией по идентификатору текущего пользователя, что исключает отображение чужих данных. Для дашборда пациента дополнительно строится запрос приёмов на текущий день с диапазонной фильтрацией по полю `scheduled_at`. Дашборд врача агрегирует данные о соблюдении курса по каждому пациенту, вычисляя процент принятых доз относительно общего числа прошедших запланированных приёмов.

Обновление курса. Пациент может редактировать только самостоятельно добавленные курсы: при наличии заполненного `doctor_id` редактирование блокируется с выводом соответствующего сообщения. Врач обновляет поле `doctor_comment` через отдельный маршрут, что позволяет добавлять клинические рекомендации к курсу без изменения его основных параметров.

Удаление курса. При удалении курса каскадные ограничения базы данных (ON DELETE CASCADE) автоматически удаляют все связанные за-

писи: позиции курса (CourseItem), запланированные и фактические приёмы (IntakeLog) и прикрепленные фотографии рецептов (PrescriptionPhoto). На уровне приложения перед удалением проверяется принадлежность курса текущему пользователю; попытка удалить чужой курс возвращает ответ 403. Пациент лишён возможности удалить курс, назначенный врачом, – соответствующая проверка поля `doctor_id` реализована в маршруте явно.

Динамическое формирование состава курса. Форма добавления курса поддерживает произвольное количество препаратов: пользователь последовательно добавляет позиции через JavaScript-интерфейс, каждая из которых включает наименование препарата, дозировку, единицу измерения, время и дни приёма. На стороне клиента данные накапливаются в скрытых полях формы в виде параллельных массивов (`medicine_name[]`, `dosage[]`, `times[]`, `days[]`) и передаются одним POST-запросом. На сервере маршрут обходит массивы в цикле и создаёт по одной записи CourseItem на каждую позицию.

Таким образом, реализованный CRUD-интерфейс обеспечивает полный цикл управления курсами лечения с разграничением операций по ролям, защитой от несанкционированного доступа к чужим данным и автоматическим каскадным удалением зависимых записей, что соответствует задаче 5, сформулированной в разделе 1.3.

3.4 Реализация системы учёта приёмов лекарств и агрегация статистики

Система учёта приёмов является центральным функциональным модулем приложения и охватывает три взаимосвязанных аспекта: хранение фактических данных о приёмах в журнале IntakeLog, отображение дневника с цветовой индикацией соблюдения расписания и агрегацию статистики для обеих ролей.

Структура журнала приёмов. Каждая запись таблицы `intake_log` соответствует одному запланированному приёму конкретного препарата и содержит поля: `course_item_id` – ссылка на позицию курса, `patient_id` – ссылка на пациента, `scheduled_at` – дата и время по расписанию, `taken_at` – фактическое время приёма, `status` – текущий статус записи. Поле `status` принимает одно из четырёх значений: `pending` (ожидает), `taken` (принято), `missed` (пропущено) или `skipped` (отмечено пользователем как пропущенное).

Записи журнала формируются автоматически функцией `generate_scheduled_logs` в момент сохранения курса (подробно рассмотрена в подразделе 3.1) и изначально имеют статус `pending`.

Алгоритм фиксации факта приёма. Пациент отмечает приём нажатием кнопки в дашборде или дневнике. POST-запрос поступает на маршрут `/patient/intake/{id}/mark`. В листинге 7 представлена реализация данного маршрута.

Листинг 7 – Маршрут фиксации факта приёма (`app/routes/patient.py`)

```
1 @patient_bp.route('/intake/<int:log_id>/mark', methods=['  
    POST'])  
2 @login_required  
3 @patient_required  
4 def mark_intake(log_id):  
5     log = IntakeLog.query.get_or_404(log_id)  
6     if log.patient_id != current_user.id:  
7         abort(403)  
8  
9     action = request.form.get('action', 'taken')  
10    log.status = action  
11    if action == 'taken':  
12        log.taken_at = datetime.utcnow()  
13    db.session.commit()  
14    next_url = request.form.get('next',  
15        url_for('patient.diary',  
16            date=log.scheduled_at.date().isoformat()))  
17    return redirect(next_url)
```

Параметр `action` передаётся из формы и принимает значение `'taken'` при нажатии кнопки «Принял» или `'skipped'` при нажатии «Пропустил». При значении `'taken'` дополнительно фиксируется время фактического приёма в поле `taken_at`, что позволяет впоследствии анализировать отклонение от запланированного времени. Перед изменением статуса проверяется, что запись принадлежит текущему пациенту, – в противном случае возвращается ответ 403. После фиксации пользователь перенаправляется на страницу, с которой был инициирован запрос, что обеспечивает сохранение контекста навигации.

Алгоритм определения статуса дня для дневника. Дневник приёмов отображает недельный мини-календарь с цветовой индикацией каждого дня.

Для каждого дня недели вычисляется обобщённый статус на основе всех записей IntakeLog, запланированных на этот день. В листинге 8 представлена реализация алгоритма классификации статуса дня.

Листинг 8 – Алгоритм классификации статуса дня (app/routes/patient.py)

```
1 for day in week_days:
2     day_logs = IntakeLog.query \
3         .join(CourseItem).join(Course) \
4         .filter(
5             Course.patient_id == current_user.id,
6             IntakeLog.scheduled_at >= datetime.combine(
7                 day, datetime.min.time()),
8             IntakeLog.scheduled_at <= datetime.combine(
9                 day, datetime.max.time())
10        ).all()
11
12     if not day_logs:
13         week_statuses[day] = 'none'
14     elif any(l.status == 'missed' or
15             (l.status == 'pending' and day < date.today())
16             for l in day_logs):
17         week_statuses[day] = 'missed'
18     elif any(l.status == 'pending' and day == date.today()
19             for l in day_logs):
20         week_statuses[day] = 'pending'
21     elif all(l.status == 'taken' for l in day_logs):
22         week_statuses[day] = 'taken'
23     elif any(l.status == 'taken' for l in day_logs):
24         week_statuses[day] = 'partial'
25     else:
26         week_statuses[day] = 'pending'
```

Алгоритм реализует систему приоритетов статусов. Наивысший приоритет имеет статус missed: день классифицируется как пропущенный, если хотя бы один приём не был отмечен до момента его наступления (прошедший день с записью в статусе pending эквивалентен пропуску). Статус partial присваивается, когда хотя бы один приём за день зафиксирован, но не все записи переведены в статус taken. Статус taken возможен только при условии, что все записи за день переведены в это состояние. Результирующий словарь week_statuses передаётся в шаблон и используется для формирования цветовой индикации ячеек мини-календаря: зелёный – принято, красный – пропущено,

жёлтый – частично, серый – ожидает.

Алгоритм обнаружения конфликтов времени приёма. При одновременном наличии нескольких курсов у пациента возможны ситуации, когда разные препараты назначены на одно и то же время. Система автоматически выявляет такие конфликты и отображает предупреждение на дашборде. В листинге 9 представлена реализация алгоритма обнаружения конфликтов.

Листинг 9 – Алгоритм обнаружения конфликтов времени приёма
(app/routes/patient.py)

```
1 def _find_conflicts(logs):
2     from collections import defaultdict
3     by_time = defaultdict(list)
4     for log in logs:
5         t = log.scheduled_at.strftime('%H:%M')
6         by_time[t].append(log)
7     conflicts = []
8     for t, ls in by_time.items():
9         if len(ls) > 1:
10            names = ', '.join(
11                f"{l.course_item.medicine.name} "
12                f"{l.course_item.dosage} "
13                f"{l.course_item.unit}"
14                for l in ls
15            )
16            conflicts.append({'time': t, 'names': names})
17     return conflicts
```

Функция группирует записи журнала приёмов за текущий день по времени (HH:MM) с помощью словаря defaultdict. Временные слоты, в которых оказывается более одного препарата, формируют список конфликтов с указанием точного времени и наименований препаратов. Термин «конфликт времени приёма» используется в значении совпадения времени приёма двух и более препаратов: система информирует пациента об этом факте, однако не блокирует приём, поскольку одновременный приём препаратов может быть намеренным решением врача или пациента. Результат передаётся в шаблон дашборда и отображается пользователю в виде предупреждающего блока.

Агрегация статистики соблюдения курса. Процент соблюдения курса вычисляется методом get_compliance() модели Course. Метод выбирает все записи журнала по позициям курса с датой не позднее текущего дня и вычисляет

отношение принятых доз к общему числу прошедших запланированных приёмов. На основе полученного процента метод `get_status_label()` классифицирует соблюдение по трём градациям: «Хорошо» ($\geq 75\%$), «Внимание» ($\geq 50\%$) и «Критично» (менее 50%), сопоставляя каждой градации соответствующий визуальный стиль в интерфейсе.

Дашборд пациента агрегирует суммарный процент соблюдения по всем активным курсам: из множества записей журнала, дата которых не превышает текущую, вычисляются счётчики принятых доз и общего числа прошедших приёмов. Страница статистики врача формирует аналогичные показатели в разбивке по каждому курсу каждого пациента посредством трёх отдельных запросов к таблице `intake_log` с фильтрацией по статусу.

Таким образом, реализованная система учёта приёмов обеспечивает автоматическое формирование расписания, фиксацию фактов приёма с временными метками, цветовую визуализацию соблюдения расписания в недельном дневнике, детектирование конфликтов времени и многоуровневую агрегацию статистики для пациентов и врачей, что соответствует задачам 6 и 7, сформулированным в разделе 1.3.

3.5 Реализация загрузки и выгрузки файлов

Функциональность работы с файлами в приложении реализована в двух направлениях: загрузка фотографий рецептов пользователем (входящие файлы) и генерация PDF-отчёта с историей приёмов для скачивания (исходящие файлы).

Конфигурация файловой подсистемы. Параметры загрузки файлов задаются в классе `Config`. Допустимые форматы хранятся в множестве `ALLOWED_EXTENSIONS`, максимальный размер файла ограничен `MAX_CONTENT_LENGTH` значением 16 МБ – Flask автоматически отклоняет запросы, превышающие этот предел, не передавая управление обработчику маршрута. Файлы сохраняются в директорию `uploads/prescriptions/`, путь к которой формируется относительно корня проекта. В листинге 10 представлены параметры конфигурации файловой подсистемы.

Листинг 10 – Конфигурация файловой подсистемы (config.py)

```
1 UPLOAD_FOLDER          = os.path.join(BASE_DIR, 'uploads', '
    prescriptions')
2 MAX_CONTENT_LENGTH     = 16 * 1024 * 1024    # 16 MB
3 ALLOWED_EXTENSIONS     = {'png', 'jpg', 'jpeg', 'gif', 'pdf'}
```

Вспомогательная функция `allowed_file` проверяет допустимость расширения загружаемого файла: она извлекает расширение после последней точки в имени файла и сравнивает его с множеством разрешённых значений в нижнем регистре. Проверка по расширению, а не по MIME-типу из заголовка запроса, устойчива к его намеренному искажению клиентом.

Загрузка фотографии рецепта. Пациент прикрепляет фотографию рецепта к позиции курса через форму с атрибутом `enctype="multipart/form-data"`. Маршрут `/patient/intake/{id}/upload_photo` обрабатывает загруженный файл. В листинге 11 представлена реализация алгоритма сохранения файла.

Листинг 11 – Алгоритм загрузки фотографии рецепта (app/routes/patient.py)

```
1 file = request.files['photo']
2 if file and allowed_file(file.filename,
3                             current_app.config['ALLOWED_EXTENSIONS']):
4     filename = secure_filename(
5         f"rx_{log.course_item_id}_"
6         f"{datetime.utcnow().strftime('%Y%m%d%H%M%S')}"
7         f"{file.filename}")
8     )
9     file.save(os.path.join(
10         current_app.config['UPLOAD_FOLDER'], filename))
11     course_item = log.course_item
12     if course_item.prescription_photo:
13         course_item.prescription_photo.file_path = filename
14         course_item.prescription_photo.uploaded_at = \
15             datetime.utcnow()
16     else:
17         db.session.add(
18             PrescriptionPhoto(course_item_id=course_item.id,
19                               file_path=filename))
20     db.session.commit()
```

Имя сохраняемого файла формируется из трёх составляющих: префикс `rx_`, идентификатор позиции курса и метка времени в формате `YYYYMMDDHHMMSS`. Функция `secure_filename` из библиотеки `Werkzeug`

нейтрализует потенциально опасные символы в исходном имени файла (обход директорий, пробелы, спецсимволы), предотвращая атаки на файловую систему. Фотография рецепта привязывается к позиции курса (CourseItem), а не к отдельной отметке приёма, поскольку рецепт подтверждает назначение препарата в целом, а не конкретный факт приёма дозы. Перед сохранением проверяется наличие существующей фотографии для данной позиции курса: если она уже есть – обновляется путь и метка времени; в противном случае создаётся новая запись PrescriptionPhoto. В базе данных хранится только относительный путь к файлу, а не его бинарное содержимое, что снижает нагрузку на СУБД.

Просмотр загруженной фотографии реализован маршрутом /patient/photo/{id}, который считывает путь к файлу из записи PrescriptionPhoto и возвращает его браузеру через функцию send_file с MIME-типом image/jpeg. Перед отдачей файла выполняется проверка принадлежности записи текущему пользователю.

Генерация и выгрузка PDF-отчёта. Экспорт истории приёмов в формате PDF реализован функцией export_history_pdf в модуле app/utls.py с использованием библиотеки ReportLab. Функция принимает объект курса и объект пациента, формирует документ в памяти сервера и возвращает путь к временному файлу.

Документ строится по модели PLATYPUS (Page Layout and Typography Using Scripts): последовательность элементов (story) передаётся в SimpleDocTemplate, который размещает их на страницах формата A4 с полями 20 мм. Документ включает заголовок с реквизитами курса и пациента, сводную строку статистики (процент соблюдения, число принятых и пропущенных доз), при наличии – комментарий врача, и таблицу всех записей журнала, отсортированных по дате.

Поскольку встроенные шрифты ReportLab (Helvetica, Times) не содержат символов кириллицы, реализован механизм регистрации Unicode-шрифта. Функция _register_cyrillic_font последовательно проверяет стандартные пути размещения шрифта DejaVuSans в операционной системе, а при его отсутствии загружает файл шрифта во временную директорию. Зарегистрированный шрифт присваивается всем стилям абзацев и ячейкам таблицы через параметр fontName, что обеспечивает корректное отображение кириллического текста в

генерируемом документе.

Передача готового файла клиенту выполняется функцией `send_file` с параметром `as_attachment=True`, что инициирует диалог скачивания в браузере. Имя скачиваемого файла задаётся параметром `download_name` и содержит идентификатор курса для однозначной идентификации. В листинге 12 представлена реализация маршрута экспорта.

Листинг 12 – Маршрут экспорта истории приёмов в PDF (`app/routes/patient.py`)

```
1 @patient_bp.route('/courses/<int:course_id>/export_pdf')
2 @login_required
3 @patient_required
4 def export_pdf(course_id):
5     course = Course.query.get_or_404(course_id)
6     if course.patient_id != current_user.id:
7         abort(403)
8     pdf_path = export_history_pdf(course, current_user)
9     return send_file(
10         pdf_path,
11         as_attachment=True,
12         download_name=f'medtreker_course_{course_id}.pdf',
13         mimetype='application/pdf'
14     )
```

Аналогичный маршрут реализован для роли врача: `/doctor/courses/{id}/export_pdf` формирует PDF по данным курса пациента и доступен только врачу, назначившему данный курс.

Таким образом, реализованная подсистема файлового обмена обеспечивает безопасную загрузку изображений рецептов с проверкой формата и нейтрализацией имён файлов, постоянное хранение на файловой системе сервера со ссылкой в базе данных и формирование PDF-документов с кириллическим содержимым для скачивания, что соответствует задаче 8, сформулированной в разделе 1.3.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсового проекта была достигнута поставленная цель: спроектировано и реализовано веб-приложение МедТрекер для учёта и контроля приёма лекарственных средств, обеспечивающее двустороннее взаимодействие между врачом и пациентом и автоматизацию анализа приверженности лечению.

В процессе работы решены все сформулированные задачи:

1. Проведён сравнительный анализ существующих программных продуктов (Medisafe, MyTherapy, «Приём лекарств», «Мои таблетки»), который показал, что ни одно из рассмотренных решений не реализует полноценную двухролевою модель «врач – пациент» с поддержкой назначения курсов, статистики для врача и экспорта отчётности. Обоснован выбор технологического стека: Python 3.13, Flask 3.1.3, SQLite 3, Bootstrap 5, Jinja2.
2. Спроектирована архитектура приложения на основе паттерна MVC с применением механизма Blueprint для разделения функциональности по ролям. Разработана модель данных, включающая шесть таблиц: user, medicine, course, course_item, intake_log, prescription_photo - с каскадными ограничениями целостности и поддержкой всех требуемых связей.
3. Разработаны макеты восьми экранов приложения методом code-based prototyping, составлена диаграмма вариантов использования с разграничением функций по ролям, сформированы пользовательские истории для обеих ролей.
4. Реализована система аутентификации на основе Flask-Login с хранением паролей в виде хэшей PBKDF2-SHA256 и двухуровневым контролем доступа: декоратор @login_required проверяет факт аутентификации, пользовательские декораторы patient_required и doctor_required - соответствие роли маршруту.
5. Разработаны модули управления справочником лекарственных средств и курсами лечения: полный CRUD-цикл с разграничением операций по ролям, автоматическим созданием позиций курса и каскадным удалением зависимых записей.

6. Реализован дневник приёма лекарств с недельным мини-календарём и цветовой индикацией статусов, автоматической генерацией расписания приёмов на весь период курса, фиксацией фактического времени отметки и возможностью прикрепления фотографий рецептов к позиции курса.
7. Реализованы автоматический расчёт процента соблюдения курса лечения с классификацией по трём градациям («Хорошо», «Внимание», «Критично») и алгоритм обнаружения конфликтов времени приёма препаратов с отображением предупреждения на дашборде пациента.
8. Обеспечен экспорт истории приёмов в формат PDF средствами библиотеки ReportLab с поддержкой кириллического текста, сводной статистикой соблюдения курса и полной таблицей записей журнала. Экспорт доступен как пациенту, так и врачу.
9. Проведено функциональное тестирование всех реализованных модулей, исходный код размещён в открытом репозитории на платформе GitHub, приложение развёрнуто на хостинге.

Разработанное приложение реализует все функциональные требования, сформулированные по результатам анализа предметной области, и устраняет ключевые недостатки существующих аналогов: отсутствие ролевого интерфейса врача, невозможность дистанционного назначения курсов и дефицит инструментов экспорта отчётности.

В качестве направлений дальнейшего развития системы можно выделить добавление push-уведомлений о времени приёма, интеграцию с внешними медицинскими базами данных лекарственных взаимодействий, а также масштабирование СУБД до PostgreSQL при увеличении числа пользователей.

Исходный код разработанного приложения размещён в открытом репозитории на платформе GitHub и доступен по адресу: <https://github.com/AnastasiaSoloveva06/course-project-web-dev>.

Приложение развёрнуто на хостинге PythonAnywhere и доступно по адресу: <https://asoloveva.pythonanywhere.com>.

СПИСОК ЛИТЕРАТУРЫ

- [1] World Health Organization. Adherence to Long-term Therapies: Evidence for Action [Electronic resource]. — [S. 1.] : Женева: ВОЗ, 2003. — URL: <https://www.who.int/publications/i/item/9241545992>. (дата обращения: 22.04.2026).
- [2] MediSafe Inc. Напоминания и трекер таблеток [Электронный ресурс]. — [Б. м. : б. и.], 2024. — URL: <https://medisafeapp.com/> (дата обращения: 22.04.2026).
- [3] smartpatient GmbH. MyTherapy: напоминания о приёме лекарств [Электронный ресурс]. — [Б. м. : б. и.], 2024. — URL: <https://www.mytherapyapp.com/> (дата обращения: 22.04.2026).
- [4] Расписание приёма лекарств с напоминаниями [Электронный ресурс]. — [Б. м. : б. и.], 2025. — URL: <https://www.rustore.ru/catalog/app/com.whisperarts.mrpillster>. (дата обращения: 22.04.2026).
- [5] Мои таблетки — Приём лекарств и напоминания [Электронный ресурс]. — [Б. м. : б. и.], 2025. — URL: <https://www.rustore.ru/catalog/app/com.wachanga.pills> (дата обращения: 22.04.2026).
- [6] Python Software Foundation. Python Release Python 3.13.0 [Electronic resource]. — [S. 1. : s. n.]. — URL: <https://www.python.org/downloads/release/python-3130/>. (дата обращения: 22.04.2026).
- [7] Pallets Projects. Flask Documentation [Electronic resource]. — [S. 1. : s. n.]. — URL: <https://flask.palletsprojects.com>. (дата обращения: 22.04.2026).
- [8] SQLite Consortium. Distinctive Features Of SQLite [Electronic resource]. — [S. 1. : s. n.]. — URL: <https://sqlite.org/different.html>. (дата обращения: 22.04.2026).
- [9] The Bootstrap Team. Bootstrap 5 Documentation [Electronic resource]. — [S. 1. : s. n.]. — URL: <https://getbootstrap.com/docs/5.0/>. (дата обращения: 22.04.2026).
- [10] Pallets Projects. Jinja2 Documentation [Electronic resource]. — [S. 1. : s. n.]. — URL: <https://jinja.palletsprojects.com>. (дата обращения: 22.04.2026).